Migration from SQL to NoSQL with performance evaluation

Hirakjyoti Talukdar
Roll No: 252210009
Dept. of CSE
National Institute of Technology Delhi
252210009@nitdelhi.ac.in

Uddipta Deka
Roll No: 252210028
Dept. of CSE
National Institute of Technology Delhi
252210028@nitdelhi.ac.in

Mayank Mukul
Roll No: 252210015
Dept. of CSE
National Institute of Technology Delhi
252210015@nitdelhi.ac.in

Raju Maurya
Roll No: 252210020
Dept. of CSE
National Institute of Technology Delhi
252210020@nitdelhi.ac.in

Dr. Shelly Sachdeva
Faculty Advisor
Dept. of CSE
National Institute of Technology Delhi
shellysachdeva@nitdelhi.ac.in

***Abstract*—The current work designs and tests a full-fledged data migration pipeline from a relational MySQL database to two different NoSQL stores, namely MongoDB and Apache Cassandra. Following the idea of DLoader migration strategy put forward by K. Rajaram et al., who propose their paper "Migration of Data from SQL to NoSQL Databases," Apache Spark is used as the main ETL tool that extracts TPC-H dataset, transforms it, and loads it to two NoSQL databases with appropriate schema mappings. The original concept has been improved in this project through the design of a performance benchmarking framework based on Apache JMeter with 15 different queries (5 queries per database) involving point queries, detail queries, aggregations, range queries, and calculation of revenue values. Experimentation proves that Apache Cassandra shows better response time in case of any query while MySQL has average performance and MongoDB lags behind due to aggregation pipelines used for complex operations on hierarchical documents.**

***Index Terms*—NoSQL, MongoDB, Cassandra, MySQL, Data Migration, TPC-H, Apache Spark, ETL Pipeline, Performance Benchmarking, JMeter, DLoader**

I. Introduction

With the increasing amount of data used by the current application, the drawbacks of traditional RDBMS systems have been revealed. Relational databases such as MySQL offer ACID properties and known SQL queries; however, they struggle with scalability issues horizontally, dynamic schemas, and perform well when there are fewer write and read operations. The new generation of NoSQL databases has come up as a solution for the problems stated above.

Migrating data from SQL databases to NoSQL databases is not an easy process; thus, it needs to be done effectively and efficiently. The DLoader framework introduced by Rajaram et al. has been developed as a solution to migrate databases using Apache Spark as the processing engine and HDFS as the intermediary storage system. This project implements the following DLoader framework:

- A complete ETL pipeline using PySpark for extracting TPC-H data from MySQL.
- Schema transformation and mapping to both MongoDB (nested documents) and Cassandra (flat denormalized rows).
- Optimized loading to MongoDB (batch insertion with Decimal128 conversion and 16 MB limit fallback) and Cassandra (Spark Cassandra Connector with tuned write parameters and idempotent truncation).
- A JMeter-based benchmarking suite with 15 queries across three databases for performance comparison.
- A custom HTML comparison report generator for visualizing results.

The TPC-H benchmark dataset is used as the source data, providing a standardized and reproducible basis for evaluating migration correctness and query performance across the three database systems.

II. Literature Review

Several approaches have been proposed for migrating data from relational to NoSQL databases. Ramzan et al. [1] proposed an intelligent data engineering approach for migration to NoSQL-based secure environments. Potey et al. [2] discussed migration from structured to non-structured databases with a focus on schema mapping challenges. Jogi and Sinha [3] evaluated the performance of MySQL, Cassandra, and HBase for heavy write operations.

The DLoader system by Rajaram et al. provides a generalized framework for SQL to NoSQL migration using Spark. It employs HDFS as a middle layer for staging data and applies column-to-column mapping from source to target databases. The system was tested on immunization data migrated to both Cassandra and MongoDB, demonstrating that Cassandra consumes 41% less storage space than MongoDB for equivalent datasets [4].

Lee and Zheng [5] addressed SQL-to-NoSQL schema denormalization, while Hamouda and Zainol [6] proposed a document-oriented schema for relational database migration. Rocha et al. [7] developed a framework for migrating relational datasets to NoSQL with automated schema conversion. Our work builds on DLoader by applying it to the industry-standard TPC-H dataset and adding comprehensive read performance benchmarking.

III. System Architecture and Methodology

A. Overall Architecture

Our Project follows a five-stage ETL pipeline architecture, as illustrated in Fig. 2. The pipeline is orchestrated by the primary execution module and consists of: (1) Extraction from MySQL, (2) Staging as local Parquet files, (3) Transformation and denormalization using Spark SQL joins, (4) Schema mapping to target NoSQL formats, and (5) Loading into MongoDB and Cassandra.

The architecture follows the DLoader paper's approach but replaces HDFS with local Parquet staging for simpler single-node deployment. Fig. 1 depicts the original DLoader extraction and loading pipeline, while Fig. 2 depicts our adapted Spark SQL framework that uses local storage and writes to both MongoDB and Cassandra.

B. Data Extraction

The extraction module creates a Spark session configured with connectors for MongoDB, Cassandra, and MySQL. JDBC extraction connects to a live MySQL instance with partitioned parallel reads. Eight TPC-H tables are extracted: `customer`, `orders`, `lineitem`, `nation`, `region`, `part`, `supplier`, and `partsupp`. For JDBC mode, each table is configured with appropriate partition columns and bounds to enable parallel extraction. For example, the `lineitem` table uses `l_orderkey` with 200 partitions across bounds [1, 60000000].

C. Data Staging

Extracted DataFrames are written as local Parquet files to staging directory. This staging step provides a checkpoint between extraction and transformation, enabling the pipeline to be restarted from the transformation phase without re-reading from the source database. This approach mirrors the HDFS staging layer described in the DLoader paper but uses the local filesystem for simpler deployment.

D. Data Transformation

The transformation stage ingests the staged Parquet files and applies a sequence of join operations to construct a fully denormalized dataset. The following joins are executed:

- `orders` LEFT JOIN `lineitem` ON `order_id` (to retain orders that have no line items)
- `customer` LEFT JOIN `orders_with_items` ON `customer_id`
- Enrichment with the `nation` and `region` dimensions through foreign key relationships

Before renaming columns, each table is processed with a `dropna()` call that eliminates rows containing nulls in essential key attributes, preserving integrity for the joins that follow. Columns are then renamed to replace the original TPC-H identifiers (e.g., `c_custkey`, `o_orderkey`) with more descriptive business names (e.g., `customer_id`, `order_id`). The result is a single wide DataFrame that consolidates all customer, order, line item, nation, and region data.

E. Schema Mapping

The mapping module transforms the denormalized DataFrame into two target representations:

*1) **MongoDB Mapping (Document Store):*** For MongoDB, the denormalized relational data is restructured into a hierarchical document model. All orders belonging to a single customer are grouped together and embedded as an array within that customer's document. Within each order, the associated line items are further nested as an array of sub-documents. This process produces a three-level nested structure where each customer document contains its orders, and each order contains its line items. Fig. 3 illustrates an example document.

*2) **Cassandra Mapping (Wide-Column Store):*** For Cassandra, the hierarchical data is flattened into a single, wide denormalized table. Each row in this table represents a unique line item, but it also contains all relevant customer and order information duplicated across rows. To efficiently retrieve all data belonging to a single customer, the table is organized such that rows for the same customer are stored together on the same physical node. This is achieved using the customer identifier as the primary means for data distribution. Within each customer's set of rows, orders and line items are sorted in order, facilitating rapid, ordered scans of a customer's complete purchase history. Table I shows the resulting Cassandra table schema.

TABLE I
Cassandra Customer_Orders Table Schema

Column	Type	Key Role
customer_id	INT	Partition Key
order_id	INT	Clustering Key
line_number	INT	Clustering Key
name	TEXT	-
nation	TEXT	-
region	TEXT	-
status	TEXT	-
total_price	DECIMAL	-
order_date	DATE	-
quantity	DECIMAL	-
extended_price	DECIMAL	-
discount	DECIMAL	-
tax	DECIMAL	-
return_flag	TEXT	-
line_status	TEXT	-
ship_date	DATE	-

Table II presents the conceptual mapping from SQL to NoSQL.

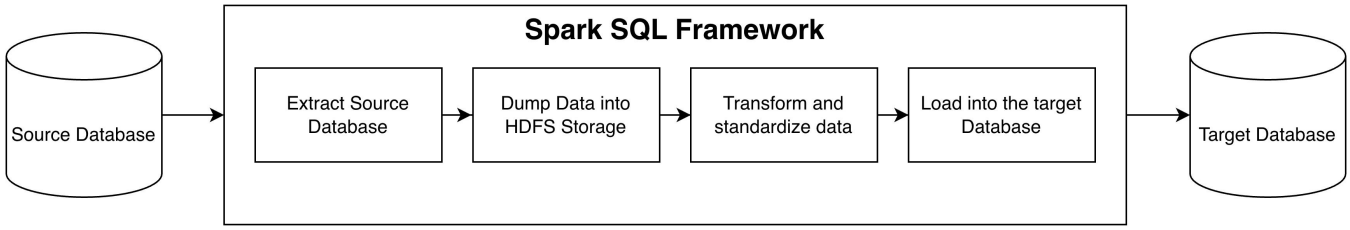

Fig. 1. DLoader Extraction and loading pipeline.

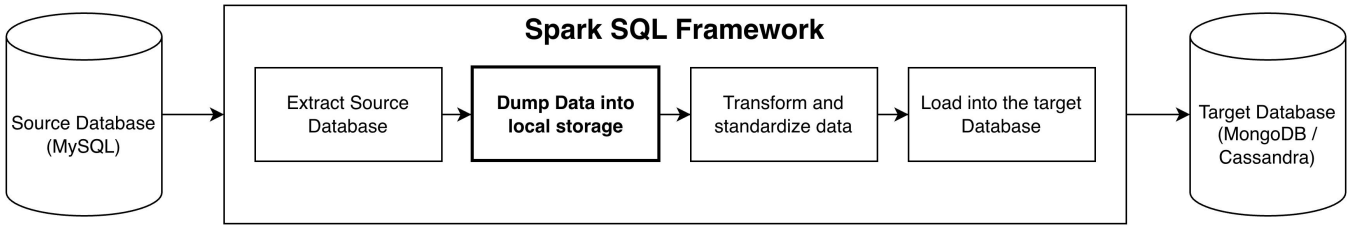

Fig. 2. Our Spark SQL framework with local storage and target databases.

```
{
  "customer_id": 1,
  "name": "Customer#1",
  "nation": "ALGERIA",
  "region": "AFRICA",
  "orders": [
    {
      "order_id": 123,
      "status": "O",
      "total_price": 12345.67,
      "lineitems": [
        {
          "line_number": 1,
          "quantity": 17,
          "extended_price": 21168.23
        }
      ]
    }
  ],
  "discount": 0.04
}
```

Fig. 3. Example MongoDB customer document structure

TABLE II
CONCEPT MAPPING: SQL TO NOSQL

SQL Concept	MongoDB Equivalent	Cassandra Equivalent
Database	Database	Keyspace
Table	Collection	Table
Row	Document (JSON)	Row
Column	Field	Column
Primary Key	ObjectId / _id	Partition + Clustering Key
JOIN	Embedded / $lookup	Denormalized Table
Index	Index	Secondary Index
Foreign Key	Reference / Embed	Not Supported

F. Data Loading

*1) **MongoDB Loading**:* To maintain compatibility with the BSON format used by MongoDB, the customers document with nested documents must go through recursive transformation before being written out as Parquet files. Transformation is needed since Spark DataFrame and PyArrow create Python objects that cannot be serialized by the PyMongo driver. In particular, there are three major transformations that are done: the `Decimal` object is transformed into `Decimal128` in order to maintain precision of decimal representation; Numpy objects like `np.int64` and `np.float64` are transformed to the built-in Python types `int` and `float`, respectively; and the Python `date` object is transformed to the Python `datetime` for the sake of compatibility with the UTC date format used in MongoDB. After the transformation, the documents are written out as intermediate Parquet files and read back with PyArrow to be inserted into the MongoDB database using the PyMongo driver. Batch operations involving 5,000 records are used, but in case a record exceeds the MongoDB 16 MB document limit, single insertions are used.

*2) **Cassandra Loading**:* Keyspace and tables are created using the native Cassandra driver for Python. The advantage of managing schemas using the driver instead of the Spark Cassandra Connector is that the former offers much better control over replication settings and definition of the table schema itself, making sure that the resulting table matches precisely the denormalized structure produced in the previous step. Loading is done with the use of the Spark Cassandra Connector but with carefully tuned write settings. The batch size is set to one record in order to completely eliminate any batch from the process, thus avoiding `Batch too large` errors that occur frequently due to large numbers of orders. To cover for the lack of batch processing, we increase the write concurrency up to ten, allowing for inserting multiple records in parallel without reducing performance. Connection timeout is increased to 120 seconds in order to account for network latency and possible compaction operations, and to avoid terminating jobs prematurely. Finally, the target table is always truncated before loading. This allows for achieving idempotence – running the same pipeline several times results in an identical output and no duplicates, which helps to maintain benchmark stability.

IV. Experimentation and Benchmarking

A. Test Environment

The experiments were conducted on the following environment:

TABLE III
TEST ENVIRONMENT SPECIFICATIONS

Component	Specification
Operating System	macOS
Java Runtime	OpenJDK 17
Framework	Apache Spark 3.5.0 (PySpark)
Source Database	MySQL 8.0 (TPC-H 1 GB dataset)
Target: MongoDB	MongoDB Community Server (localhost)
Target: Cassandra	Apache Cassandra (localhost, datacenter1)
Benchmarking Tool	Apache JMeter 5.6.3
Language	Python 3.9+, Groovy (JMeter scripts)
Driver Memory	6 GB (Spark driver and executor)

B. TPC-H Dataset

The TPC-H benchmark is an industry-standard decision support benchmark consisting of 8 tables with complex relationships. For this project, a 1 GB scale factor dataset was used, generating approximately 150,000 customers, 1.5 million orders, and 6 million line items. The five core tables used in migration are: `customer`, `orders`, `lineitem`, `nation`, and `region`.

C. Benchmark Design

A custom JMeter test plan (`benchmark_nosql.jmx`) was created with 15 queries organized into five categories, each implemented across all three databases using external Groovy scripts.

TABLE IV
BENCHMARK QUERY DESCRIPTIONS

Query	Type	Description
Q1	Point Query	Read a single customer by primary/partition key
Q2	Detail Query	Read all orders and line items for a customer
Q3	Aggregation	Count orders grouped by status for a customer
Q4	Range/Filter	Filter customers by region or return flag
Q5	Revenue	Compute total revenue across line items

Each database is tested in a separate thread group running sequentially (MongoDB, Cassandra, MySQL) to avoid cross-database interference. The test configuration uses 5 concurrent threads with 100 loop iterations per thread, resulting in 500 samples per query per database (7,500 total samples). A teardown thread group closes all database connections after benchmarking.

V. Results and Analysis

A. Performance Results

The benchmark was executed with 5 threads and 100 loops per thread, generating 500 samples per query. All queries completed with 0% error rate. The results are summarized in Table V and Table VI.

TABLE V
PERFORMANCE RESULTS (5 THREADS × 100 LOOPS)

Query	Avg Response Time (ms)			Samples
	MongoDB	Cassandra	MySQL	
Q1 - Point Query	46.1	3.2	6.8	500
Q2 - Detail Query	76.3	2.3	17.7	500
Q3 - Aggregation	71.6	1.8	9.2	500
Q4 - Range/Filter	77.3	2.1	8.1	500
Q5 - Revenue	82.3	2.8	35.5	500

TABLE VI
OVERALL THROUGHPUT AND LATENCY SUMMARY

Database	Avg Latency (ms)	Min (ms)	Max (ms)	Throughput (req/s)	Error %
Cassandra	2.4	0	539	431.6	0.0%
MySQL	15.5	0	329	92.5	0.0%
MongoDB	70.7	1	1607	14.8	0.0%

B. Analysis of Results

*1) **Cassandra Performance**:* Cassandra achieves the best performance across all query types with average response times ranging from 1.8 ms to 3.2 ms. Its overall average latency of 2.4 ms and throughput of 431.6 requests per second far exceed the other databases. This superior performance is attributed to its partition-key based data model, which enables O(1) lookups when the partition key (`customer_id`) is provided. All data for a given customer is co-located on disk,

making partition scans extremely efficient and minimizing disk I/O.

*2) **MySQL Performance**:* Throughput in the MySQL database is moderate, with the average time spent to perform operations ranging between 6.8 ms and 35.5 ms, the mean being 15.5 ms per operation, at a rate of 92.5 queries per second. Simple lookups (`Q1`, `Q4`) leverage index access via primary keys and execute relatively quickly, whereas complex queries that involve `JOIN`s across multiple tables (`Q2`, `Q5`) tend to exhibit much higher latencies. The longest latency among MySQL operations belongs to the revenue aggregation query (`Q5`), which takes 35.5 ms, mainly because of the `JOIN`s involved.

*3) **MongoDB Performance**:* Among the three databases, MongoDB records the highest latencies, with per-query response times spanning 46.1 ms to 82.3 ms and an overall mean of 70.7 ms at a throughput of just 14.8 requests per second. The elevated cost originates from the overhead of its aggregation framework, which is tightly coupled to the nested document structure. Operations that deconstruct arrays with `$unwind` and then recombine data via `$group` stages are particularly expensive. Even straightforward point lookups (Q1) deliver relatively weak performance at 46.1 ms, and more elaborate aggregation queries (Q3–Q5) all surpass 70 ms.

VI. Tools and Technologies

Table VII lists the main tools and technologies used in this project.

TABLE VII
TOOLS AND TECHNOLOGIES

Tool / Technology	Version	Purpose
Apache Spark / PySpark	3.5.0	ETL processing and data transformation
MySQL	8.0	Source relational database (TPC-H)
MongoDB	Community	Target document-oriented NoSQL database
Apache Cassandra	5.0.8	Target wide-column NoSQL database
Apache JMeter	5.6.3	Performance benchmarking and load testing
Python	3.9+	Pipeline scripting and orchestration

VII. Challenges and Solutions

Several technical challenges were encountered during implementation:

A. MongoDB 16 MB Document Size Limit

Customers with very large numbers of orders produced documents exceeding MongoDB's 16 MB BSON limit. Solution: Implemented fallback one-by-one insertion with graceful skipping of oversized documents.

B. Cassandra Write Timeouts and Batch Size Errors

Bulk loading via the Spark Cassandra Connector caused 'Batch too large' and write timeout exceptions. Solution: Disabled batching (`batch.size.rows = 1`), increased concurrent writes to 10, and extended connection timeouts to 120 seconds.

C. Connection Management Under Concurrency

JMeter's multi-threaded execution required thread-safe database connection handling. Solution: Implemented singleton connection patterns using JMeter's `props` object with synchronized blocks for lazy initialization of MongoDB, Cassandra, and MySQL connections.

VIII. Conclusion

This project successfully implements the DLoader methodology for migrating data from MySQL to both MongoDB and Cassandra using the TPC-H benchmark dataset. The five-stage ETL pipeline (Extract, Stage, Transform, Map and Load) provides a clean, flexible architecture for data migration with schema-appropriate conversions for each target database.

The benchmarking results in 7,500 query samples confirm the following key findings:

- Cassandra provides the best read performance (avg 2.6 ms) due to partition-key optimized data access.
- MySQL offers moderate performance (avg 14.9 ms) with indexed relational queries.
- MongoDB shows a higher latency (avg 68.6 ms) due to the aggregation pipeline overhead on nested documents.
- All three databases achieved 0% error rate, confirming the data integrity across migrations.

These results align with the original DLoader paper's conclusion that Cassandra offers higher performance for read-heavy workloads on denormalized data. The wider performance gap observed in our experiments (compared to the original paper) can be attributed to the more complex nested document structure used for MongoDB and the broader range of query types that we tested.

IX. Future Work

- Evaluate the write performance (INSERT, UPDATE, DELETE) across all the three databases.
- Test with larger TPC-H scale factors (10 GB, 100 GB) to analyze scalability.
- Implementing graph database (Neo4j) as additional migration target.
- Add index optimization analysis for MongoDB compound indexes.
- Deploy it on a multi-node cluster to evaluate distributed performance.
- Implement automated data validation between the source and target databases.

REFERENCES

[1] S. Ramzan, I. S. Bajwa, B. Ramzan, and W. Anwar, "Intelligent data engineering for migration to NoSQL based secure environments," *IEEE Access*, vol. 7, pp. 69042–69057, 2019.

[2] M. Potey, M. Digrase, G. Deshmukh, and M. Nerkar, "Database migration from structured database to non-structured database," *Int. J. Comput. Appl.*, vol. 975, no. 8887, 2015.

[3] V. D. Jogi and A. Sinha, "Performance evaluation of MySQL, Cassandra and HBase for heavy write operation," in *Proc. IEEE RAIT*, 2016, pp. 586–590.

[4] K. Rajaram et al., "DLoader: Migration of Data from SQL to NoSQL Databases," in *Lecture Notes in Networks and Systems*, Springer, 2023.

[5] C. H. Lee and Y. L. Zheng, "SQL-to-NoSQL schema denormalization and migration," in *Proc. IEEE SMC*, 2015, pp. 2022–2026.

[6] S. Hamouda and Z. Zainol, "Document-oriented data schema for relational database migration to NoSQL," in *Proc. IEEE Innovate-Data*, 2017, pp. 43–50.

[7] L. Rocha, F. Vale, E. Cirilo, and D. Barbosa, "A framework for migrating relational datasets to NoSQL," *Procedia Comput. Sci.*, vol. 51, pp. 2593–2602, 2015.